

Figure 2: The dynamic sparsity of each layer and head in Llama-3-8B model in the KV retrieval test of 100,000 tokens. The blue curve shows that using dynamically selected top-1000 critical tokens achieves an average recovery ratio of 89%, indicating high attention sparsity. In contrast, the orange curve reveals that statically using the initially determined top-1000 critical tokens from the generation of the first token to generate subsequent tokens drops the average recovery ratio to 71%.

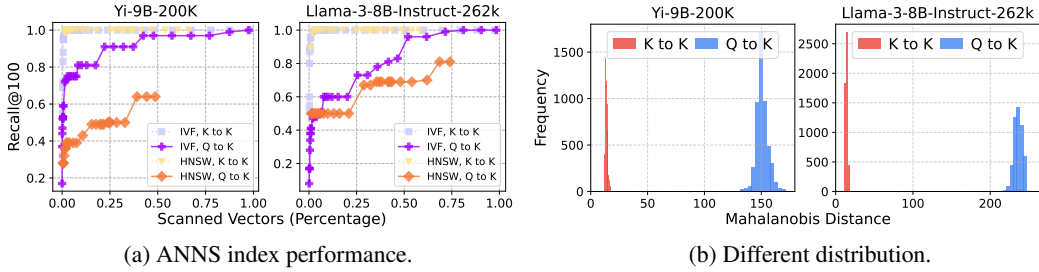


Figure 3: (a) Query vectors (Q) and key vectors (K) are dumped from Yi-9B and Llama-3-8B with a prompt length of 128,000 tokens. Off-the-shelf ANNS indexes perform poorly on Q to K searches while work well for K to K searches. (b) Query vectors are distant from key vectors, while key vectors themselves are close.

Furthermore, we observe that as the LLM continues generating new tokens, the critical key vectors change dynamically, highly depending on the current query vector. To verify this, we first collect the top-1000 critical key vectors to generate the first token in each attention head and statically apply them for the subsequent token generation. The results shown in the orange curve of Figure 2 indicate a significant drop in the average recovery rate, from 89% to 71%. This demonstrates that tokens considered important in previous queries may not be critical in subsequent queries, and vice versa. Therefore, it is necessary to dynamically select important tokens for each query vector.

The dynamic sparsity shows a promising path to approximately compute attention with greatly reduced cost and without sacrificing the model accuracy. For each query, if we can accurately identify the relevant key-value vectors with higher importance, minimum GPU memory and a much lower time complexity can be achieved for attention computation.

2.4 CHALLENGES OF OFF-THE-SHELF VECTOR SEARCH

To reduce the latency of long contexts inference while maintaining performance, we require a method to accurately identify the critical tokens to the current query in sub-linear time complexity. Additionally, given the constrained GPU memory, it would be beneficial if such a method could efficiently utilize CPU memory to manage the KV vectors. Based on Equation 1, one key vector is critical for a query vector if they have a large inner product. With the inner product as a similarity function, performing searches on ANNS indexes aligns well with the goal of the attention mechanism to efficiently find critical key vectors for a query vector.

Traditional ANNS indexes generally cluster similar (close) vectors and select the representative vector for each cluster (Sivic & Zisserman, 2003) or directly build connections between similar vectors to form a proximity graph (Wang et al.).[†] For cluster-based indexes, the query first compares with all representative vectors and then only accesses the most similar clusters, whereas, in the proximity

[†]In this context, we use “similar” and “close” to indicate vectors with larger inner product interchangeably.

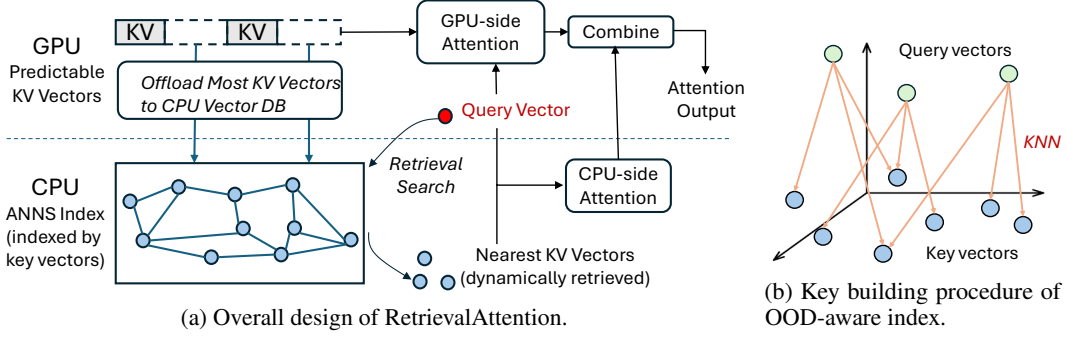


Figure 4: (a) RetrievalAttention offloads most KV tokens to vector databases in CPU, which are retrieved during the decoding phase to find the most relevant KV tokens with queries. (b) During the index construction, we link each query to its exact top- k nearest key vectors (KNN).

graph, the query performs a greedy search, moving closer to the most similar vectors at each hop. Both methods typically require scanning a limited subset of all vectors (e.g., 1%) to identify the most similar vectors to the query, achieving high search efficiency and accuracy. However, we find that naively applying off-the-shelf vector indexes fails to provide good performance because of the OOD issue between query (Q) and key vectors (K).

In conventional vector databases, the distribution of vectors between content and query is often well-aligned because they are derived from the same embedding model. However, naively using traditional vector indexes for attention computation suffers from an inherent distribution gap between queries and keys, which are projected by different weights as 2.1. Figure 3a (focus on Q to K for now) demonstrates the performance of widely-used vector indexes supported by Faiss (Douze et al., 2024) using a query vector to retrieve the most similar key vectors. It compares the percentage of keys scanned and the corresponding recall achieved (i.e., the overlapping ratio between the retrieved top-100 results and the ground truth). Cluster-based IVF (Sivic & Zisserman, 2003) requires scanning ~ 30 – 50% data for a recall rate higher than 0.95, and graph-based HNSW (Malkov & Yashunin, 2018) falls into a local optimum. The results show that traditional vector indexes require scanning a large number of vectors to achieve a high recall, highlighting the challenge of performing efficient vector searches for attention.

Fundamentally, the difficulty is due to the OOD between query and key vectors. We quantify this using Mahalanobis distance (Mahalanobis, 2018), which measures the distance from a vector to a distribution. We sample 5,000 vectors from Q and K respectively as the query set and compute the Mahalanobis distance from the query set to the remaining vectors in K . Figure 3b shows that the queries from Q are significantly distant from the K vectors (OOD) while K themselves are very close. Therefore, traditional index building based solely on the closeness between key vectors does not align with the attention mechanism, which requires to retrieve critical tokens as nearest neighbors from the query vectors’ viewpoint. In contrast, Figure 3a shows that using sampled K as the queries (K to K) can easily achieve a high recall by only scanning 1–5% vectors because they are in the same distribution. Similarly, query vectors in each attention head also follow the same distribution as they are projected by the same model weight. For efficient vector search, the index must consider the OOD characteristic of the attention computation by design.

3 RETRIEVALATTENTION DESIGN

In this work, we focus on the acceleration of token generation and assume the prefill of the long-context prompt is done in advance, which is widely supported by existing LLM service providers (e.g., context caching (Google Cloud, 2024) or separation of prefill and decoding (Patel et al., 2024; Qin et al., 2024)).

We propose RetrievalAttention that leverages attention-aware vector search to approximate attention computation by CPU-GPU co-execution accurately. Figure 4a shows the overall design of RetrievalAttention. Based on our observation in §2.3, We derive an approximated attention by

selectively retrieving relevant key-value vectors while discarding those that are negligible (§3.1). To efficiently support long context, we offload most KV vectors to the CPU memory, build vector indexes, and use attention-aware vector search to find critical tokens. (§3.2). To better exploit the GPU devices, we leverage the attention scores obtained in the prefill phase to select a proportion of KV cache that is consistently important during the decoding phase and persist them on GPU devices. RetrievalAttention computes partial attention with dynamically retrieved from CPU memory and persistent key-value vectors in GPU memory in parallel and finally combines them together (§3.3).

3.1 APPROXIMATED ATTENTION

Based on the Equation 1, RetrievalAttention approximates the full attention output $\mathbf{o}_t$ by selectively utilizing the KV vectors associated with high attention scores (*i.e.*, $a_{t,i}$). Specifically, we define $\mathcal{I}_{t,\epsilon}$ as a subset of token indices for which the attention score surpasses ϵ . Consequently, a sparse attention mechanism, which only considers tokens located in $\mathcal{I}_{t,\epsilon}$, can be defined as follows:

$$\mathbf{o}_t = \sum_{i \in \mathcal{I}_{t,\epsilon}} a_{t,i} \cdot \mathbf{v}_i + \sum_{i \notin \mathcal{I}_{t,\epsilon}} a_{t,i} \cdot \mathbf{v}_i \approx \sum_{i \in \mathcal{I}_{t,\epsilon}} \tilde{a}_{t,i} \cdot \mathbf{v}_i \quad \text{where} \quad \tilde{a}_{t,i} = \frac{e^{z_i}}{\sum_{j \in \mathcal{I}_{t,\epsilon}} e^{z_j}} \quad (2)$$

Based on the above approximation, we build RetrievalAttention to only consider important key-value vectors (*i.e.*, $\mathcal{I}_{t,\epsilon}$) that are persistent in GPU cache and dynamically retrieved by vector indexes.

3.2 ATTENTION-AWARE VECTOR SEARCH

For each pair of key and value vectors, we first decide whether to hold them in CPU or GPU memory (the decision method is elaborated in §3.3). The KV vectors offloaded to CPU memory will be indexed by $\mathbf{k}_i \in \mathcal{R}^d$ and queried by $\mathbf{q}_t$ to find the most relevant ones.

To accelerate the vector search during token generation, RetrievalAttention diverges from traditional indexes that only consider the closeness between key vectors for index building. Instead, it leverages the existing query vectors in the prefill phase to guide the index building for key vectors, efficiently mitigating the distribution gap. As shown in Figure 4b, during the index construction, RetrievalAttention explicitly establishes connections from the query vector to its nearest key vectors (*i.e.*, exact k -nearest neighbors, or KNN). The KNN results can be efficiently computed via GPU, forming a mapping from query vector distribution to key vector distribution. Using this structure, the decoding query vector can first search its nearest query vectors and then obtain the most relevant key vectors through the distribution mapping.

Therefore, the KNN connections from query vectors to key vectors serve as a bridge to reconcile their distribution differences. However, this structure still has imperfections in both memory overhead and search efficiency because we need to store and access query vectors besides key vectors. To address this problem, we leverage the projection technique from the state-of-the-art cross-modal ANNS index RoarGraph (Chen et al., 2024a) to eliminate all query vectors. Specifically, we project the KNN connections into key vectors by linking key vectors that are connected to the same query vectors, which efficiently streamlines the search. This process connects key vectors that are perceived as close from the query vectors' perspective, allowing efficient index traversal for future query vectors.

Our evaluation shows that, by effectively modeling the proximity relationship between the query and key vectors, the vector database only requires scanning 1–3% key vectors to reach a high recall, significantly reducing the index search latency by 74% compared with IVF indexes (Sivic & Zisserman, 2003).

3.3 CPU-GPU CO-EXECUTION

To exploit GPU parallelism and accelerate attention computation, RetrievalAttention decomposes the attention computation into two disjoint sets of KV cache vectors, the predictable ones on GPU and the dynamic ones on CPU, and then combines the partial attention outputs together.

We leverage the patterns observed in the prefill phase to predict KV vectors that are consistently activated during token generation. Similar to StreamingLLM (Xiao et al., 2024b), our current implementation uses fixed initial tokens and the last sliding window of the context as the static pattern